

HABIT TRACKER ARCHITECTURE BRIEF

Technical & Product Design Questionnaire Response

Author: Maruthi Benhur

Date: May 29, 2026

Role: React Native Developer
Candidate

Question 1 — Streak Logic and Behavioral Design

A streak is a powerful cognitive motivator, but its mathematical implementation can feel jarring to users if not aligned with daily habits. I define a streak as: the number of consecutive calendar days a habit has been completed, terminating either today or yesterday.

Streak Retention Rules:

- **Active Streak (Today Completed):** If a habit is marked done today, the streak includes today and counts backward consecutively.
- **Pending Streak (Today Incomplete):** If today is not marked done yet, but yesterday was completed, the streak remains active at its yesterday-value. This displays a "pending" status. The user has until the calendar day ends to complete the habit.
- **Broken Streak (Yesterday Missed):** If both today and yesterday are incomplete, the streak immediately resets to 0.

Specific Edge Cases:

- **Missed Yesterday, Completed Today:** If a user forgets to log yesterday, today's completion resets the streak. Today becomes Day 1 of a new streak. The previous streak is broken.
- **Opened After 3 Days:** If the app is opened after 3 days of total inactivity, the streak shows as 0. Completing it today starts a new streak of 1.

Why This Approach?

I chose this over a strict "today-only" definition (where streaks show as 0 in the morning until completed). Showing 0 at the start of a day is psychologically penalizing; it makes the user feel like their yesterday's progress was erased. Preserving the streak as "pending" for today acts as a nudge rather than a punishment, maintaining engagement while strictly enforcing the 24-hour reset if the day actually closes without log completion.

Question 2 — Data Model and AsyncStorage Limits

For a simple offline-first habit tracker, the storage model is designed for quick updates.

Data Model:

Each habit record is stored as an object within a root habits list:

```
{
  "id": "1716954203000",
  "name": "Drink 3L Water",
  "createdAt": "2026-05-29T08:15:00.000Z",
  "history": ["2026-05-26", "2026-05-27", "2026-05-28", "2026-05-29"]
}
```

This array of habits is serialized into a single JSON string and stored locally under the AsyncStorage key "@habits_data".

Retrieval & Rendering:

1. Read the JSON string from AsyncStorage and parse it into an array of Habit objects.
2. Find the selected habit by its unique ID.
3. Compute the date strings for the last 7 calendar days relative to the local system time: [D-6, D-5, ..., D].
4. Convert the habit's history array into a Set for O(1) lookups, then map over the 7 days to check completion.

Scalability Failures (30 Habits, 2 Years of History):

With 30 habits and a 70% completion rate over 2 years, the total completions count is ~15,000 entries. This is small (~350 KB) and works fine in AsyncStorage, but it will break in three ways:

1. Thread Blocking: JSON parsing is synchronous. Parsing a multi-megabyte string blocks React Native's single JavaScript thread, causing screen transition lags and startup delays.
2. Write Amplification: Every time a checkbox is toggled, the entire history is rewritten to the device's storage. Toggling one cell writes hundreds of kilobytes, draining battery and wearing out storage.
3. Memory Footprint: Loading years of history into memory when the user only needs today's status or a 7-day grid is highly inefficient.

Question 3 — The Single Most Important Engineering Improvement

If I had one more day, the single most important improvement I would make is migrating local storage from AsyncStorage to SQLite (using "expo-sqlite") and normalizing the schema.

Why This Above Everything Else?

While improving the UI or adding features like notification reminders is tempting, fixing the data layer is the most critical engineering task. A habit tracker is built on consistency over months and years. A database design that slows down as the user succeeds is a fundamental product flaw.

Implementing a relational database with two normalized tables solves our scalability issues:

1. "habits" table: holds metadata (id, name, created_at)
2. "habit_logs" table: holds a row for each completion (habit_id, date_string) with a unique composite index.

Under this model:

- Toggling a habit is O(1): We run an atomic SQL query: INSERT OR IGNORE INTO habit_logs... or DELETE FROM habit_logs... This modifies a single row rather than rewriting the entire habits JSON file.
- Queries are highly efficient: We retrieve only the last 7 days of logs (using: SELECT date_string FROM habit_logs WHERE habit_id = ? AND date_string >= D-6), completely decoupling rendering speed from the lifetime size of the logs database.
- Reliable data integrity: SQLite prevents duplicate entries and handles concurrent writes safely, preventing data loss or corruption which can occur with raw file storage.